Week 6 Lab 1: Implement Clean Architecture

Purpose

In Week 5, the application gained relational and document persistence. In this lab, you will refactor one existing use case into a Clean Architecture structure.

The main architectural rule is the **Dependency Rule**:

```
Source-code dependencies point towards Business and Domain
```

You will implement the same use case through two incoming adapters:

- MVC and Razor
- REST and JSON

Both adapters will invoke the same Business use case and use the same Business-owned persistence gateway.

Core Scope

This two-hour core lab focuses only on:

```
W5-UC01: View Student Enrolments
```

Use the existing description in:

```
week05-lab1-use-cases.md
```

Preserve its:

- Primary actor.
- Main success scenario.
- **No Enrolments Found** alternative flow.
- Earliest-to-latest ordering rule.
- Required display fields.
- Read-only postconditions.

The independent refactoring of:

```
W5-UC04: Record Research Activity
```

is an extension task. It is not required for core completion within the two-hour session.

Learning Outcomes

By completing the core lab, you should be able to:

- Implement one use case across Domain, Business, Persistence, Web, API, and Application projects.
- Use an input port and a Business-owned output port to preserve the Dependency Rule.
- Separate Domain entities, Business DTOs, persistence records, MVC view models, and API response models.
- Invoke the same Business use case through MVC and REST adapters.

Files and Projects Required

Before starting, make sure you have:

- Your implementation of week 5.
- The supplied Week 6 starter solution.
- `week05-lab1-use-cases.md`.
- `week06-lab1-initial-class-diagram.md`.

The supplied starter already contains the project scaffold:

```
PgrStudentManagement.sln
├── PgrStudentManagement.Domain/
├── PgrStudentManagement.Business/
├── PgrStudentManagement.Adapters.Persistence/
├── PgrStudentManagement.Adapters.Web/
├── PgrStudentManagement.Adapters.API/
├── PgrStudentManagement.Application/
└── PgrStudentManagement.Tests/
```

It also contains the required project references and basic host configuration.

You do not need to recreate the projects during the core lab.

About the Initial Class Diagram

The supplied class diagram provides an architectural starting point for `W5-UC01`.

It intentionally does not define every final property, mapping decision, method signature, or implementation relationship.

Use it to identify:

- The project responsible for each class.
- The input port used by incoming adapters.
- The output port required by Business.
- The direction of source-code dependencies.
- The separation between Domain, persistence, MVC, and API models.

During Lab 2, you will revise and complete the diagram so that it represents your actual implementation.

Part 1: Verify the Starter

Task 1: Inspect and Build the Six-Project Scaffold

From the solution directory, run:

```
dotnet sln list
dotnet restore
dotnet build
```

Confirm that the solution contains:

```
PgrStudentManagement.Domain
PgrStudentManagement.Business
PgrStudentManagement.Adapters.Persistence
PgrStudentManagement.Adapters.Web
PgrStudentManagement.Adapters.API
PgrStudentManagement.Application
PgrStudentManagement.Tests
```

Inspect the project references and verify the dependencies direction:

```
Business
    -> Domain

Persistence
    -> Business
    -> Domain

Web
    -> Business

API
    -> Business

Application
    -> Business
    -> Persistence
    -> Web
    -> API
```

A Web or API reference to Domain is permitted only where the implemented adapter genuinely requires a Domain type. Do not add unnecessary project references.

Confirm that these references do not exist:

```
Business -> Persistence
Business -> Web
Business -> API
Web -> Persistence
API -> Persistence
Web -> API
API -> Web
```

Run the scaffolded host:

```
dotnet run --project PgrStudentManagement.Application
```

Confirm that `PgrStudentManagement.Application` is the only executable host.

Create a checkpoint before changing the code:

```
git status
git add .
git commit -m "Preserve Week 6 starter scaffold"
```

Part 2: Implement the Domain and Business Layers

Task 2: Create or Move the Domain Enrolment Model

In the Domain project, create or complete:

```
PgrStudentManagement.Domain/
└── Students/
    ├── Student.cs
    ├── Enrolment.cs
    └── EnrolmentStatus.cs
```

The Domain classes represent business concepts. They must not contain:

- Entity Framework Core attributes.
- MongoDB or BSON attributes.
- MVC validation attributes.
- JSON serialization attributes.
- `DbContext` or `DbSet<T>` references.
- Controller, view, API, or persistence-record references.

A representative `Enrolment` is:

```
namespace PgrStudentManagement.Domain.Students;

public class Enrolment
{
    public int EnrolmentId { get; init; }

    public string StudentNumber { get; init; } = string.Empty;

    public string ProgrammeCode { get; init; } = string.Empty;

    public DateTime StartDate { get; init; }

    public DateTime? ExpectedEndDate { get; init; }

    public string ModeOfStudy { get; init; } = string.Empty;

    public EnrolmentStatus Status { get; init; }
}
```

The example and supplied diagram show only the properties relevant to the guided use case. Do not remove existing Student state or behaviour required by earlier use cases.

If the implementation stores only `StudentNumber` on `Enrolment`, model that identifier-based relationship accurately. Do not add a `Student.Enrolments` collection unless the Domain design for your implementation genuinely uses it.

Build the Domain project:

```
dotnet build PgrStudentManagement.Domain
```

Task 3: Create the Business Output Port

Create:

```
PgrStudentManagement.Business/
└── Students/
    └── Gateway/
        └── IEnrolmentGateway.cs
```

Add:

```
using PgrStudentManagement.Domain.Students;

namespace PgrStudentManagement.Business.Students.Gateway;
```

```
public interface IEnrolmentGateway
{
    Task<IReadOnlyList<Enrolment>> GetForStudentAsync(
            string studentNumber,
            CancellationToken cancellationToken);
}
```

`IEnrolmentGateway` is an **output port**.

It expresses the external data capability required by Business. It belongs to Business because the use case owns that requirement.

The interface must not mention:

- EF Core.
- `DbContext`.
- Persistence records.
- SQLite.
- MongoDB.
- MVC or API types.

Task 4: Create the Business Input Port and DTOs

Create:

```
PgrStudentManagement.Business/
└── Students/
    └── ViewStudentEnrolments/
        ├── IViewStudentEnrolmentsUseCase.cs
        ├── ViewStudentEnrolmentsUseCase.cs
        ├── ViewStudentEnrolmentsInput.cs
        ├── EnrolmentOutput.cs
        └── StudentEnrolmentsOutput.cs
```

Use-case DTOs contain simple data crossing the Business boundary.

A possible input DTO is:

```
namespace PgrStudentManagement.Business.Students.ViewStudentEnrolments;

public record ViewStudentEnrolmentsInput(string StudentNumber);
```

A possible output structure is:

```
public record EnrolmentOutput(
    string ProgrammeCode,
```

```
    DateTime StartDate,
    DateTime? ExpectedEndDate,
    string ModeOfStudy,
    string Status);

public record StudentEnrolmentsOutput(
    string StudentNumber,
    IReadOnlyList<EnrolmentOutput> Enrolments);
```

Create the input port:

```
public interface IViewStudentEnrolmentsUseCase
{
    Task<StudentEnrolmentsOutput> ExecuteAsync(
        ViewStudentEnrolmentsInput input,
        CancellationToken cancellationToken);
}
```

`IViewStudentEnrolmentsUseCase` is an **input port**. It defines how MVC and REST adapters invoke the use case.

Do not add the following to Business DTOs:

- MVC display or validation attributes.
- EF Core attributes.
- BSON attributes.
- HTTP status codes.
- Razor-specific formatting.

Task 5: Implement and Test the Business Use Case

Implement `ViewStudentEnrolmentsUseCase` using constructor injection of `IEnrolmentGateway`.

The use case should:

1. Receive `ViewStudentEnrolmentsInput`.
2. Call `IEnrolmentGateway`.
3. Order matching Enrolments from earliest to latest `StartDate`.
4. Map Domain Enrolments to Business output DTOs.
5. Return an empty Enrolments collection for the supplied **No Enrolments Found** flow.
6. Avoid modifying any Student or Enrolment information.

The chronological ordering rule belongs to the Business use case because it is observable use-case behaviour.

The persistence gateway may also order its database query for efficiency, but the use case must remain correct if a gateway returns records in a different order.

Add the required test references if they are not already present:

```
dotnet add PgrStudentManagement.Tests \
    reference PgrStudentManagement.Business

dotnet add PgrStudentManagement.Tests reference PgrStudentManagement.Domain
```

Create:

```
PgrStudentManagement.Tests/
├── TestDoubles/
│   └── FakeEnrolmentGateway.cs
└── Business/
    └── ViewStudentEnrolmentsUseCaseTests.cs
```

The fake gateway should implement `IEnrolmentGateway` without EF Core or SQLite.

Add test cases for:

- Matching Enrolments are returned.
- Unrelated Enrolments are excluded.
- Results are ordered from earliest to latest even when the fake returns an unsorted collection.
- A Student with no Enrolments produces an empty output collection.
- Viewing Enrolments does not modify the fake data.

Run:

```
dotnet test
```

These Business tests must not require:

- EF Core.
- SQLite.
- ASP.NET Core MVC.
- A running web server.

Part 3: Implement the Relational Persistence Adapter

Task 6: Create the EF Records and Persistence Mapper

In the Persistence Adapter, create or move:

```
PgrStudentManagement.Adapters.Persistence/
└── Relational/
    ├── Models/
    │   ├── StudentRecord.cs
```

```
│   └── EnrolmentRecord.cs
├── Configurations/
├── Mappers/
│   └── EnrolmentPersistenceMapper.cs
├── Gateway/
│   └── EfEnrolmentGateway.cs
└── StudentManagementDbContext.cs
```

The Week 6 starter should already contain the selected .NET 8-compatible package versions. Do not upgrade the EF Core packages to a different major version.

If a package must be restored manually, use the same `8.0.x` patch version already selected for every EF Core package in the solution.

The persistence records represent the relational schema. They may contain:

- Primary keys.
- Foreign keys.
- EF navigation properties.
- EF configuration.
- Database-specific details.

They are not Domain entities.

Create a read mapper containing only the operation required by the guided use case:

```
public Enrolment ToDomain(EnrolmentRecord record)
{
    // Map the EF record to the Domain entity.
}
```

Do not add a `ToRecord` operation unless another implemented use case requires it.

Preserve the existing table and column names. Configure records explicitly where necessary, for example:

```
modelBuilder.Entity<StudentRecord>()
    .ToTable("Student");

modelBuilder.Entity<EnrolmentRecord>()
    .ToTable("Enrolment");
```

Also preserve existing keys, foreign keys, indexes, nullability, and enum conversions.

Task 7: Implement the EF Gateway

Implement:

```
EfEnrolmentGateway : IEnrolmentGateway
```

The gateway should:

1. Query `EnrolmentRecord` objects using `StudentManagementDbContext`.
2. Use `AsNoTracking()` because the use case is read-only.
3. Filter using the supplied student number.
4. Order by `StartDate` for query efficiency.
5. Map every persistence record to a Domain `Enrolment`.
6. Return an empty list when no records match.

A representative implementation is:

```
public async Task<IReadOnlyList<Enrolment>>
    GetForStudentAsync(
        string studentNumber,
        CancellationToken cancellationToken)
{
    var records = await _context.Enrolments
        .AsNoTracking()
        .Where(record =>
            record.StudentNumber == studentNumber)
        .OrderBy(record => record.StartDate)
        .ToListAsync(cancellationToken);

    return records
        .Select(_mapper.ToDomain)
        .ToList();
}
```

The runtime call reaches `EfEnrolmentGateway`, but Business has a source-code dependency only on `IEnrolmentGateway`.

Verify the Existing Database Mapping

The Persistence Adapter contains the `DbContext` and migrations. The Application project supplies startup configuration and the connection string.

If the starter already runs against the existing Week 5 database, do not create a new migration unnecessarily.

If a migration is required, use:

```
dotnet ef migrations add CleanArchitectureMapping \
    --project PgrStudentManagement.Adapters.Persistence \
    --startup-project PgrStudentManagement.Application
```

Apply migrations using:

```
dotnet ef database update \
    --project PgrStudentManagement.Adapters.Persistence \
    --startup-project PgrStudentManagement.Application
```

Before applying a generated migration, inspect it. Do not apply a migration that unintentionally drops and recreates the existing Student or Enrolment tables.

Part 4: Implement the Incoming Adapters

Task 8: Implement the MVC and Razor Adapter

In the Razor Class Library, create or complete:

```
PgrStudentManagement.Adapters.Web/
├── Controllers/
│   └── StudentsController.cs
├── ViewModels/
│   ├── StudentEnrolmentsViewModel.cs
│   └── EnrolmentViewModel.cs
├── Mappers/
│   └── StudentEnrolmentsWebMapper.cs
└── Views/
    └── Students/
        └── Enrolments.cshtml
```

The MVC controller depends on:

```
IViewStudentEnrolmentsUseCase
StudentEnrolmentsWebMapper
```

It must not depend on:

- `StudentManagementDbContext`.
- `EfEnrolmentGateway`.
- EF persistence records.
- The API adapter.

The controller action should be asynchronous:

```
public async Task<IActionResult> Enrolments(
    string studentNumber,
```

```
    CancellationToken cancellationToken)
```

The controller should:

1. Create `ViewStudentEnrolmentsInput`.
2. Call the input port.
3. Map `StudentEnrolmentsOutput` to the MVC view model.
4. Return the Razor view.

The Web mapper converts:

```
StudentEnrolmentsOutput -> StudentEnrolmentsViewModel
```

The MVC model may contain presentation-ready strings, including:

```
Not recorded
```

for a missing expected end date.

The Razor view must preserve:

- Required Enrolment fields.
- Earliest-to-latest ordering.
- The **No enrolments found** response.
- Read-only behaviour.

Confirm that the Web adapter uses the Razor Class Library SDK and includes the required `_ViewImports.cshtml`, `_ViewStart.cshtml`, and shared layout configuration.

Do not copy the Razor view into the Application project.

Task 9: Implement the REST API Adapter

In the API project, create or complete:

```
PgrStudentManagement.Adapters.API/
├── Controllers/
│   └── StudentsApiController.cs
├── Responses/
│   ├── StudentEnrolmentsResponse.cs
│   └── EnrolmentResponse.cs
└── Mappers/
    └── StudentEnrolmentsApiMapper.cs
```

If the scaffold does not already contain the ASP.NET Core framework reference, add this to the API project file:

```
<ItemGroup>
  <FrameworkReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

Use attribute routing:

```
[ApiController]
[Route("api/students")]
public class StudentsApiController : ControllerBase
{
    [HttpGet("{studentNumber}/enrolments")]
    public async Task<
        ActionResult<StudentEnrolmentsResponse>>
        GetEnrolments(
            string studentNumber,
            CancellationToken cancellationToken)
    {
        // Create the Business input DTO.
        // Execute the shared use case.
        // Map the output DTO to the API response.
    }
}
```

The API controller depends on the same input port as the MVC controller:

```
IViewStudentEnrolmentsUseCase
```

The API mapper converts:

```
StudentEnrolmentsOutput -> StudentEnrolmentsResponse
```

Do not return:

- MVC view models.
- Domain entities.
- EF persistence records.

For the core exercise, use:

```
200 OK with an empty Enrolments array
```

when no Enrolments match. This preserves the supplied use case's empty-result behaviour and keeps it distinct from transport-specific errors.

Part 5: Configure and Verify the Application

Task 10: Configure the Composition Root

Use `PgrStudentManagement.Application` as the only executable project.

The Application host should already reference:

```
PgrStudentManagement.Business
PgrStudentManagement.Adapters.Persistence
PgrStudentManagement.Adapters.Web
PgrStudentManagement.Adapters.API
```

Register the Business input port and Persistence output-port implementation:

```
builder.Services.AddScoped<
    IViewStudentEnrolmentsUseCase,
    ViewStudentEnrolmentsUseCase>();

builder.Services.AddScoped<
    IEnrolmentGateway,
    EfEnrolmentGateway>();
```

Register the required mappers and `StudentManagementDbContext`.

Load controllers from both incoming adapter assemblies:

```
builder.Services
    .AddControllersWithViews()
    .AddApplicationPart(
        typeof(StudentsController).Assembly)
    .AddApplicationPart(
        typeof(StudentsApiController).Assembly);
```

Configure conventional MVC routes and attribute-routed API controllers:

```
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern:
        "{controller=Home}/{action=Index}/{id?}");

app.MapControllers();
```

Keep the former Week 5 host temporarily only while moving and verifying functionality.

Do not run two active hosts.

After the new Application host works, remove the old host from the final solution or retain it only in a clearly labelled backup branch.

Task 11: Verify W5-UC01 Through MVC and REST

Run:

```
dotnet build
dotnet test
dotnet run --project PgrStudentManagement.Application
```

Test the MVC route implemented by your controller. A typical route is:

```
/Students/Enrolments?studentNumber=S100002
```

Test the REST route:

```
/api/students/S100002/enrolments
```

Also test a known Student with no Enrolments.

Confirm that both adapters:

- Invoke the same `IViewStudentEnrolmentsUseCase` input port.
- Return the same Enrolment facts.
- Preserve earliest-to-latest ordering.
- Implement the **No Enrolments Found** behaviour consistently.
- Avoid direct access to the Persistence Adapter.

Confirm that:

- Domain contains no EF, MongoDB, MVC, or JSON annotations.
- Business references Domain only.
- Business has no EF Core, MongoDB, or ASP.NET Core package dependency.
- Web and API do not reference Persistence.
- Web and API do not reference one another.
- `PgrStudentManagement.Application` is the only executable host.
- The application still runs as one process and one deployment.

Core Completion Checklist

Project Boundaries

- [ ] The six production projects build.
- [ ] Project references follow the Dependency Rule.
- [ ] `PgrStudentManagement.Application` is the only executable project.
- [ ] Adapter projects do not reference one another.

Domain and Business

- [ ] Domain entities contain no framework or persistence annotations.
- [ ] Business owns `IViewStudentEnrolmentsUseCase`.
- [ ] Business owns `IEnrolmentGateway`.
- [ ] Business DTOs contain no UI, HTTP, or persistence concerns.
- [ ] The use case owns chronological ordering.
- [ ] Business tests pass using a fake gateway.

Persistence Adapter

- [ ] EF records are separate from Domain entities.
- [ ] `EfEnrolmentGateway` implements `IEnrolmentGateway`.
- [ ] The persistence mapper converts EF records to Domain entities.
- [ ] Existing table and column mappings are preserved.
- [ ] The gateway performs a read-only query.

Incoming Adapters

- [ ] MVC and REST invoke the same Business input port.
- [ ] MVC uses a separate view model and Web mapper.
- [ ] REST uses separate response models and an API mapper.
- [ ] Neither incoming adapter accesses `DbContext` or `EfEnrolmentGateway` directly.
- [ ] The Razor view renders from the Web adapter project.

Behaviour

- [ ] The MVC page displays the required Enrolment fields.
- [ ] The REST endpoint returns the same Enrolment facts.

- [ ] Results are ordered from earliest to latest.
- [ ] A missing expected end date is represented consistently.
- [ ] A known Student with no Enrolments produces the empty-result response.
- [ ] Viewing Enrolments does not modify data.

Extension Tasks

Extension 1: Independently Refactor W5-UC04

Apply the same architecture to:

```
W5-UC04: Record Research Activity
```

Use the existing description in:

```
week05-lab2-use-cases.md
```

Create:

```
Domain
    ResearchActivity
    ResearchActivityType

Business
    IRecordResearchActivityUseCase
    RecordResearchActivityUseCase
    Input and output DTOs
    IResearchActivityGateway

Persistence
    ResearchActivityDocument
    MongoResearchActivityGateway
    ResearchActivityDocumentMapper

Web
    MVC controller action
    Form view model
    Web mapper
    Razor form

API
    REST controller
    Request and response models
    API mapper
```

The Domain entity must not contain BSON annotations.

The Business use case should:

- Validate the application-specific rules.
- Reject unsupported activity types.
- Assign the application-defined string identifier.
- Assign the creation timestamp.
- Call `IResearchActivityGateway` only after successful validation.

Use a fake gateway to test:

- Valid activity creation.
- Required information missing.
- Unsupported activity type.
- Identifier and creation timestamp assignment.
- No persistence call after failed validation.

Extension 2: Refactor Another Existing Use Case

Choose one:

```
W5-UC03: View Student Research Activities
W4-UC04: Assign Academic Supervisor to Student
W4-UC05: View Student Supervisory Team
```

Define Business ports rather than accessing another adapter or `DbContext` directly.

Extension 3: Publish and Inspect the Deployment

Run:

```
dotnet publish PgrStudentManagement.Application \
    --configuration Release
```

Identify the assemblies for:

- Domain.
- Business.
- Persistence Adapter.
- Web Adapter.
- API Adapter.
- Application host.

Explain why several assemblies still form one deployed application.

Reflection Questions

1. Which project owns `IEnrolmentGateway`, and why?
2. What is the difference between the input port and the output port?
3. How can runtime control reach the EF gateway when Business does not reference Persistence?
4. Why are Domain entities separate from EF records?
5. Why do MVC and REST use different boundary models?
6. Which classes and behaviour are shared by the MVC and REST paths?
7. What would have to change if SQLite were replaced?
8. How do project references help enforce the Dependency Rule?

Keeping the Project for Lab 2

Retain:

- The complete multi-project solution.
- The supplied initial class diagram.
- Your final implementation of `W5-UC01`.
- Business tests and fake gateway.
- Any persistence tests completed.
- Exact class, method, namespace, project, and route names.
- Notes describing deliberate differences from the initial diagram.

In Lab 2, you will create:

- A project dependency diagram.
- A C4 Component diagram.
- An implementation class diagram.
- An MVC sequence diagram.
- A REST sequence diagram.

Save a working checkpoint:

```
git status
git add .
git commit -m "Complete Week 6 Clean Architecture core refactoring"
```